AI now helps write the software that runs machines. No mainstream language can prove what that software will never do.

Batteries, flywheels and flight controls run on code that people test and review, and then trust.

Mentl is a programming language whose compiler proves it.

Mentl proves what software can never do.

You write a rule into the code, such as "this never touches the internet" or "this never writes a file".

The compiler checks the rule through every function the code calls. **Break it, and the program won't build.**

Seven months and 2,600 commits since March. The compiler is written in Mentl and compiles itself. Open source: github.com/ampactor-labs/mentl

✕ Refused

One line broke the rule.

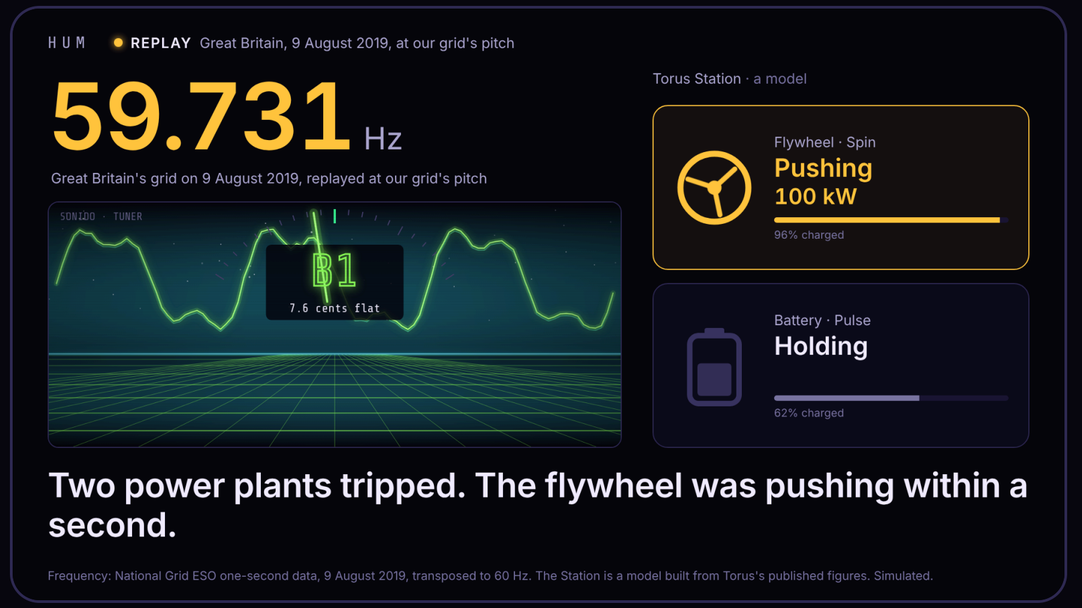
```
let price = price_m("https://api.eia.gov/…")
```

This decision is marked as never reaching the internet. Add one line that reads a price online, and the compiler won't build it. Its error names the path the price lookup took.

Show it working

Built this week: software that measures the power grid from a wall's wiring.

- **A guitar pickup picks up the grid's hum** in any room, and my audio library, sonido, measures its frequency to a thousandth of a hertz.
- **A model of a Torus Station decides** what to do, typically in under a millisecond, with software written in Mentl.
- **That software is proven** unable to reach the internet while it decides.



Before Monday 4 PM: the language, 2,610 commits since March 15, and sonido, 461 commits since January 25. Since then: everything on this slide.

Who needs it in Utah

TORUS · SALT LAKE CITY

Builds flywheel-and-battery Stations that steady the grid. The software inside each Station is where a proof would matter first. Torus judges this demo at 5:40 today.

HILL AFB · 309TH SOFTWARE ENGINEERING GROUP

Maintains flight software for the F-16, the F-35 and the missile fleet, where every claim about the software needs evidence.

First conversation I'm asking for: Torus engineering, this month.
First hires: a controls engineer and a compiler engineer, in Salt Lake City.

What $10,000 unlocks

$10,000 puts a proven controller on a real machine in Utah within ninety days.

Day 30 Form the Utah company. Run the controller on a battery inverter on my bench.

Day 60 Run it beside a real battery at one Utah site.

Day 90 A pilot agreement with that site, and an SBIR proposal with Nucleus Grow.

$7,000 for the inverter and battery bench. **$3,000** to form the company, register for federal grants and install at the site.

Two asks. A Utah operator who will host it. An introduction to Hill AFB's 309th.

BACKUP

What is proven, exactly

THE CONTROLLER'S RULE	STATUS
No network, file, clock or other call outside the program while it decides	Proven by the compiler through every call
No memory allocated while it decides	Proven
The flywheel within 100 kW, the battery within 300 kW	Proven: each machine answers a command with a power inside its rating, and the compiler checks the answer
The decision never stops with an error	Proven. One library call in the loop around it, the one that copies the timestamp, is not proven yet
Its decisions are good	Tested on recordings, not proven

The compiler is written in Mentl, and the pinned binary rebuilds itself byte for byte. The first four rows are checked through every call, and a copy that breaks one does not build.

BACKUP

From the laptop to the pedal: the day-30 engineering

WHAT STANDS IN THE WAY	MEASURED TODAY	THE FIX
No WebAssembly engine on the pedal's chip	The controller is a 37 KB WebAssembly file	Embed an interpreter written in Rust, or compile the file to C
Built for threads	Shared memory and atomic instructions	A single-threaded build; the controller uses no threads
Memory	Fixed: the same peak after 300,000 readings as after one	Done; each reading runs in an arena that is reclaimed when it ends
Speed	About 291,000 instructions per reading	Pass numbers instead of text

The decision allocates nothing and the loop reclaims each reading's memory. The work left is the engine on the chip.